Engineering Certified JSON Derivers for Rocq with ELPI

Will Thomas

University of Kansas

Abstract. Rocq developments often need executable serializers for data exchange, testing, and extracted applications, but hand-written serializers are tedious and easy to disconnect from their intended inverse. We present an ELPI-based deriver for JSON that automatically generates `Jsonifiable` instances for a practical fragment of Rocq inductive and record types. Each generated instance contains an encoder, a decoder, and a proof that decoding the encoded value succeeds with the original value. The contribution is a performance-conscious and easily usable engineering tool for mapping Rocq data to a JSON interchange format, while preserving an explicit correctness theorem.

1 Introduction

Serialization code sits at an awkward boundary in proof assistant projects. It is usually mundane enough that users do not want to write it by hand, but it is important enough that a bug can invalidate testing infrastructure or extracted tools. In Rocq, this tension is especially visible for data types that are defined primarily for proofs and then reused in extracted programs.

`rocq-json` addresses this problem with the following typeclass:

```
Class Jsonifiable (A : Type) := {
  to_JSON   : A → JSON;
  from_JSON  : JSON → Result A string;
  canonical_jsonification : forall (a : A), from_JSON (to_JSON a) = res a
}.
```

The deriver described here generates all three components for many ordinary inductive and record types. It is implemented in coq-elpi and exposed through the command `Elpi derive.jsonifiable T`.

This is a tool paper about useful engineering rather than a claim of novel foundations. The interesting part is the integration: ELPI inspection of Rocq declarations, generated code that remains suitable for extraction, typeclass parameters for nested serializable fields, and automatic proof construction for the round-trip theorem. The same ingredients are broadly useful for other derivers that construct computational content and accompanying correctness proofs.

2 Tool Overview

The user-facing interface is deliberately small. Given a declaration such as an enumeration, algebraic data type, parameterized container, or record, the user invokes the deriver and receives a registered typeclass instance. Nullary constructors are encoded as JSON strings. Constructors with arguments are encoded as singleton JSON objects whose key is the constructor name and whose value stores named fields. Records are encoded as JSON objects keyed by projection names.

For example, the following declaration and command are enough to produce an encoder, decoder, proof, and registered instance:

```
Inductive UserRole : Type :=
| Admin
| Moderator (level : nat)
| StandardUser (id : nat) (username : string)
| Guest.
Elpi derive.jsonifiable UserRole.
```

The generated representation uses a string for nullary constructors and an object for constructors with fields:

```
Example role_roundtrip (r : UserRole) :
  from_JSON (to_JSON r) = res r := canonical_jsonification r.

Example moderator_json :
  to_JSON (Moderator 2) =
    JSON_Object [("Moderator", JSON_Object [("level", JSON_Nat 2)])] :=
  eq_refl.
```

For parameterized data types, the generated instance abstracts over `Jsonifiable` instances for the type parameters. For recursive types, the generated encoder and decoder use structurally recursive definitions. The current implementation also supports a conservative collection of nested recursive occurrences through standard data containers such as lists, options, and products. More general nested or mutually recursive types are reported as unsupported rather than handled partially by ad hoc guesses.

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).
Elpi derive.jsonifiable BinTree.
Check BinTree_Jsonifiable :
  forall A, Jsonifiable A → Jsonifiable (BinTree A).
```

The generated proof is part of the public contract. A successful derivation does not merely produce functions with plausible behavior; it registers a `canonical_jsonification` proof. This property is intentionally one-sided. It says that values produced by the encoder can be decoded back to the original

value. It does not claim that every accepted JSON value is in a canonical printed form.

3 Implementation

The deriver follows the shape of the Rocq declaration. ELPI first reads the inductive or record view, classifies parameters and constructor arguments, and then builds separate skeletons for the encoder, decoder, proof, and final typeclass instance. The skeletons are elaborated by Rocq before being added as constants. This staging has been useful in practice: ELPI performs structural analysis and code generation, while Rocq remains responsible for checking the terms that become part of the trusted development.

Two engineering choices are central. First, field serializers are kept abstract through typeclass arguments where possible. This makes generated instances compose with existing hand-written or derived instances, including standard containers. Second, decoder generation is performance-sensitive. Large enumerations are decoded using a string decision tree that extracts to direct pattern matching over characters, avoiding a long chain of string equality tests. The resulting extracted code is close to the shape a careful user would write by hand.

The proof-generation strategy is mixed. General recursive and record cases use Rocq proof automation specialized to the generated encoders and decoders. Pure enumerations use a direct generated proof term, avoiding proof-search overhead on a common stress case. This division is not theoretically deep, but it matters for predictable tool latency.

4 Evaluation

We evaluate two questions: how broadly the deriver applies to existing Rocq declarations, and whether generated extracted code is competitive with hand-written code on a stress case.

4.1 Applicability

The artifact includes a benchmark script that scans installed Corelib and Stdlib sources for inductive-like declarations, probes each candidate in isolation, records diagnostics, and then compiles a single benchmark file containing all successful derivations. This workflow is deliberately conservative: unsupported declarations remain visible as categorized failures.

The category names in this table are not meant to declare every failure "invalid input". They are a triage device. Some categories are semantic non-targets for this version of the tool, such as propositions or coinductive streams. Others are engineering limitations or possible tool failures that should be treated as future work. Table 4 gives one representative declaration for each category and the rationale used by the artifact.

Table 1. Corelib/Stdlib applicability summary.

Metric	Count
Discovered declarations	343
Probed declarations	343
Successful probes	78
Probe failures	265
Probe timeouts	0
Timed successful derivations	78

Table 2. Successful derivations by declaration shape and library.

Shape	Count	Library	Count
Inductive	47	Corelib	39
Variant	17	Stdlib	39
Record/Structure	14		

Table 3. Failure categories reported by the probe benchmark.

Category	Count
prop-target-not-supported	167
dependent-field-not-supported	48
indexed-family-not-supported	21
scanner-or-logical-path	18
function-field-not-supported	4
proof-field-not-supported	3
sort-field-not-supported	3
coinductive-not-supported	1

Table 4. Representative failure examples and classification rationale.

Category	Count	Example	Rationale
prop-target-not-supported	167	Corelib.Classes.CMorphisms.apply_subrelation	Targets live in Prop, while Jsonifiable builds computational data in Type.
dependent-field-not-supported	48	Corelib.Init.Specif.sig	A constructor field type depends on an earlier constructor argument, so a plain JSON decoder cannot reconstruct the dependent evidence.
indexed-family-not-supported	21	Corelib.Init.Datatypes.relation	The declaration is a family indexed by values, not a plain data type after parameters.
scanner-or-logical-path	18	Stdlib.FSets.FMapAVL.Raw.bench	The benchmark scanner found a source declaration whose guessed logical path is not directly derivable.
function-field-not-supported	4	Corelib.ssr.ssrbool.implies	A constructor field is function-valued, and there is no generic JSON encoding for arbitrary functions.
proof-field-not-supported	3	Corelib.Init.Specif.sumbool	A constructor field lives in Prop or SProp, so the ordinary data round-trip contract would require proof erasure or proof reconstruction.
sort-field-not-supported	3	Corelib.Classes.RelationClasses.list_Type	A constructor field is itself a sort such as Type or Set, while Jsonifiable serializes data values rather than type-level objects.
coinductive-not-supported	1	Stdlib.Streams.Streams.Streams	The declaration is potentially infinite data, while the deriver targets inductive data.

The important distinction is between unsupported-by-design and unresolved. The paper claims the former only for cases where the current `Jsonifiable` interface has no clear computational target, for example `Prop` targets or potentially infinite coinductive data. The remaining unsupported categories name the blocking shape directly: dependent fields, indexed families, proof/SProp fields, sort-valued fields, or function fields. The `scanner-or-logical-path` category is a limitation of the benchmark harness. All categories are reported explicitly so the success rate is auditable rather than inflated by silent filtering.

4.2 Derivation Time

The combined successful benchmark records Rocq's `Time` output for each derived instance. In the generated run used for this draft, the combined benchmark compiled successfully with wall-clock time 5.12 seconds; the sum of Rocq-reported derivation transactions was 4.35 seconds. The mean derivation time was 0.056 seconds, the median was 0.035 seconds, the 90th percentile was 0.111 seconds, and the maximum was 0.564 seconds.

Table 5. Slowest successful derivations in the broad benchmark.

Declaration	Kind	Seconds
Corelib.Init.Byte.byte	Inductive	0.564
Stdlib.btauto.Reflect.formula	Inductive	0.180
Stdlib.rtauto.Rtauto.proof	Inductive	0.176
Stdlib.micromega.ZMicromega.ZArithProof	Inductive	0.163
Corelib.Floats.FloatClass.float_class	Variant	0.155
Stdlib.micromega.RingMicromega.Psatz	Inductive	0.153
Stdlib.Sorting.Heap.Tree	Inductive	0.145
Stdlib.setoid_ring.Field_theory.FExpr	Inductive	0.113

4.3 Extracted Code Performance

The extraction benchmark compares generated definitions against handwritten Rocq definitions after both have been extracted to OCaml. This keeps the comparison in the same source language and extraction pipeline. The benchmark covers several shapes: a 256-constructor enumeration, a tuple-like constructor, a flat record, two mixed sums with nullary and field-bearing constructors, and a recursive binary tree. Each row reports `JSONIFIABLE_EXTRACTION_RUNS` samples, which defaults to 10. The enum benchmark serializes and deserializes every constructor; the other benchmarks run many iterations over representative values. These benchmarks are still small, but they exercise different generated-code paths and are more informative than a single stress case.

The intended conclusion is not that these microbenchmarks predict every serializer workload. Rather, they check that the deriver does not impose an obvious

Table 6. Extracted round-trip benchmarks.

Benchmark	Samples	Mean	Median	Min	Max
enum256_generated	10	0.042716	0.041911	0.040905	0.047295
enum256_handwritten	10	0.042939	0.042664	0.041016	0.049128
point2d_generated	10	0.001006	0.000966	0.000877	0.001302
point2d_handwritten	10	0.001134	0.001093	0.000995	0.001508
userrole_generated	10	0.001736	0.001708	0.001628	0.001995
userrole_handwritten	10	0.002499	0.002430	0.002377	0.002733
serverconfig_generated	10	0.000618	0.000589	0.000567	0.000761
serverconfig_handwritten	10	0.001475	0.001443	0.001374	0.001666
instruction_generated	10	0.001680	0.001678	0.001579	0.001799
instruction_handwritten	10	0.002574	0.002548	0.002487	0.002689
bintree_generated	10	0.003335	0.003302	0.003010	0.003749
bintree_handwritten	10	0.004969	0.004976	0.004527	0.005391

extraction penalty on common shapes and on a case that stresses constructor dispatch.

5 Artifact

The artifact is part of the contribution. Tool papers benefit from evaluation that reviewers can rerun without reconstructing informal commands from prose. Our artifact provides a Docker path and a native path. The main script rebuilds the project from a clean state, runs the broad applicability benchmark, runs the extraction benchmark, and regenerates the LaTeX macros included in this paper.

The generated files are meant to be read directly. The JSON summary gives the aggregate result, the CSV files contain row-level data, and the Markdown failure report explains each failure category with representative Rocq diagnostics. This setup makes the boundary of the tool explicit: reviewers can see both what works and what is outside the supported fragment.

6 Limitations

The current deriver targets computational data in `Type` or `Set`. It does not attempt to serialize arbitrary propositions, dependent families, coinductive values, function fields, or records whose field types depend on earlier fields. Some nested recursion through standard containers is supported, but general nested and mutually recursive types remain outside the current design. This is a deliberate usability choice: partial support for a few accidental nested forms can be worse than a clear diagnostic when users cannot predict which structurally similar declarations will work.

The JSON representation is also intentionally modest. The paper evaluates the derivation method and the certified round-trip property, not complete adherence to the JSON RFC or interoperability with every external JSON parser.

7 Conclusion

`rocq-json`'s ELPI deriver demonstrates that useful certified serialization support can be built with a small user-facing interface and an honest artifact. The generated instances combine executable encoders and decoders with Rocq-checked invertibility proofs, apply across a meaningful fragment of existing library declarations, and extract to competitive code on a large-enumeration stress test. The engineering is specific to JSON, but the pattern is broader: use ELPI to inspect declarations, generate computational content and proof obligations together, and evaluate both applicability and runtime behavior as first-class artifact outputs.